Banco de Dados III

Introdução ao NoSQL e MongoDB

Luiz Otávio Toratti

Escola Técnica Estadual Comendador João Rays
Barra Bonita - SP

20 de fevereiro de 2026

A Evolução dos Dados: O Contexto dos Anos 70/80

Figura: IBM 3081 - Norsk Teknisk Museum

- Armazenamento (HD) era **extremamente caro** (1MB de HD custava milhares de dólares).

A Evolução dos Dados: O Contexto dos Anos 70/80

Figura: IBM 3081 - Norsk Teknisk Museum

- Armazenamento (HD) era **extremamente caro** (1MB de HD custava milhares de dólares).
- Redundância de dados era um pecado capital.

A Evolução dos Dados: O Contexto dos Anos 70/80

Figura: IBM 3081 - Norsk Teknisk Museum

- Armazenamento (HD) era **extremamente caro** (1MB de HD custava milhares de dólares).
- Redundância de dados era um pecado capital.
- Edgar Codd (IBM) propôs o Modelo Relacional.

Características da Era Relacional

- **Linguagem Padrão:** SQL (Structured Query Language)

Características da Era Relacional

- **Linguagem Padrão:** SQL (Structured Query Language)
- **Schema on Write:** Você precisa definir a tabela *antes* de salvar qualquer dado.
  - Se o dado não casar com a tabela, dá erro.

Características da Era Relacional

- **Linguagem Padrão:** SQL (Structured Query Language)
- **Schema on Write:** Você precisa definir a tabela *antes* de salvar qualquer dado.
  - Se o dado não casar com a tabela, dá erro.
- **Resultado:** Alta consistência, mas rigidez extrema.
  - Adicionar uma nova coluna em uma tabela pode travar o banco por horas.

Características da Era Relacional

- **Linguagem Padrão:** SQL (Structured Query Language)
- **Schema on Write:** Você precisa definir a tabela *antes* de salvar qualquer dado.
  - Se o dado não casar com a tabela, dá erro.
- **Resultado:** Alta consistência, mas rigidez extrema.
  - Adicionar uma nova coluna em uma tabela pode travar o banco por horas.
- **Foco:** Transações financeiras, ERPs, cadastros estáticos.

Características da Era Relacional

- **Linguagem Padrão:** SQL (Structured Query Language)
- **Schema on Write:** Você precisa definir a tabela *antes* de salvar qualquer dado.
  - Se o dado não casar com a tabela, dá erro.
- **Resultado:** Alta consistência, mas rigidez extrema.
  - Adicionar uma nova coluna em uma tabela pode travar o banco por horas.
- **Foco:** Transações financeiras, ERPs, cadastros estáticos.
- **Escala Vertical:** O banco roda em uma única máquina central (Servidor).
  - Upgrade de CPU, RAM e armazenamento.
  - Caro e limitato fisicamente.
  - Ponto único de falha.

A Evolução dos Dados: Web 2.0 e Big Data (2000+)

Com a Web 2.0 três fatores (características do big data) quebraram os bancos relacionais tradicionais:

A Evolução dos Dados: Web 2.0 e Big Data (2000+)

Com a Web 2.0 três fatores (características do big data) quebraram os bancos relacionais tradicionais:

1. **Volume:** Quantidade de dados gerados
   - Pentabytes de dados. Uma única máquina não aguenta mais guardar tudo (Fim da Escala Vertical).
   - Impacto do "User Generated Content"(UGC).

A Evolução dos Dados: Web 2.0 e Big Data (2000+)

Com a Web 2.0 três fatores (características do big data) quebraram os bancos relacionais tradicionais:

1. **Volume:** Quantidade de dados gerados
   - Pentabytes de dados. Uma única máquina não aguenta mais guardar tudo (Fim da Escala Vertical).
   - Impacto do "User Generated Content"(UGC).
2. **Velocidade:** A velocidade com que os dados chegam e precisam ser processados.
   - Milhões de usuários simultâneos (Streaming, E-commerce, IoT, ...).
   - Bancos relacionais focam em ACID (travam a tabela para garantir segurança e integridade).

A Evolução dos Dados: Web 2.0 e Big Data (2000+)

Com a Web 2.0 três fatores (características do big data) quebraram os bancos relacionais tradicionais:

1. **Volume:** Quantidade de dados gerados
   - Pentabytes de dados. Uma única máquina não aguenta mais guardar tudo (Fim da Escala Vertical).
   - Impacto do "User Generated Content"(UGC).
2. **Velocidade:** A velocidade com que os dados chegam e precisam ser processados.
   - Milhões de usuários simultâneos (Streaming, E-commerce, IoT, ...).
   - Bancos relacionais focam em ACID (travam a tabela para garantir segurança e integridade).
3. **Variedade:** Os dados possuem formatos variados.
   - Temos vídeos, imagens, audios, logs, texto livre e grafos.

A Era NoSQL (Not Only SQL)

- **Hardware Barato:** Não tem problema duplicar dados para ganhar velocidade (Desnormalização).

A Era NoSQL (Not Only SQL)

- **Hardware Barato:** Não tem problema duplicar dados para ganhar velocidade (Desnormalização).
- **Schema on Read:** Salve o dado como ele vier. Preocupe-se com o formato na hora de ler.

A Era NoSQL (Not Only SQL)

- **Hardware Barato:** Não tem problema duplicar dados para ganhar velocidade (Desnormalização).
- **Schema on Read:** Salve o dado como ele vier. Preocupe-se com o formato na hora de ler.
- **Replica Nativa:** A maioria dos NoSQL já nasce distribuída.

A Era NoSQL (Not Only SQL)

- **Hardware Barato:** Não tem problema duplicar dados para ganhar velocidade (Desnormalização).
- **Schema on Read:** Salve o dado como ele vier. Preocupe-se com o formato na hora de ler.
- **Replica Nativa:** A maioria dos NoSQL já nasce distribuída.
- **Auto-Healing:** Se o nó principal cair, os outros "elegem" um novo líder em segundos, automaticamente.

A Era NoSQL (Not Only SQL)

- **Hardware Barato:** Não tem problema duplicar dados para ganhar velocidade (Desnormalização).
- **Schema on Read:** Salve o dado como ele vier. Preocupe-se com o formato na hora de ler.
- **Replica Nativa:** A maioria dos NoSQL já nasce distribuída.
- **Auto-Healing:** Se o nó principal cair, os outros "elegem" um novo líder em segundos, automaticamente.
- **Escala Horizontal:** O banco roda em diversos servidores.
  - Adicionar mais máquinas (nós).
  - Barato e ilimitado.
  - Alta disponibilidade (Cluster).

Tipos de NoSQL

Key-Value

Document

Wide-Column

Graph

Introdução ao MongoDB: Tabela vs Documento

- Mudança fundamental na estrutura de armazenamento:

SQL

Tabela fixa com colunas pré-definidas.

ID	Nome	Email
1	Ana Silva	ana@email.com
2	Beto Pereira	null

MongoDB

Documentos BSON independentes.

```
{
  "_id": 1,
  "nome": "Ana Silva",
  "email": "ana@email.com",
  "interesses": ["Tech", "NoSQL"],
  "ativo": true
}
// O proximo documento pode ser
    diferente!
```

Relacional vs. Documento

Modelo Relacional (SQL)

- Foco no armazenamento (Normalização).
- Dados divididos para evitar duplicidade.
- "Onde guardo isso?"
- Schema rígido.

Modelo de Documento (NoSQL)

- Foco no acesso (Desnormalização).
- Dados unidos para ganhar performance.
- "Como vou ler isso?"
- Schema flexível.
- Polimorfismo.
- Suporta arrays e embedding.

- A transição exige parar de pensar em "Tabelas"e começar a pensar em "Objetos de Negócio".
- "Dados acessados juntos devem ser armazenados juntos."

Carga de Trabalho (Workload)

Modelar para MongoDB é um processo reverso. Primeiro entendemos a aplicação, depois o banco.

1. **Identificação de Entidades:** Quais são os objetos? (Curso, Aula, Aluno).
2. **Quantificação de Entidades:** Qual o "tamanho" da relação?
   - 1 Curso : 20 Aulas (1 para Poucos).
   - 1 Curso : 100.000 Alunos (1 para Muitos).

Devemos mapear a frequência e o volume das operações (Leitura e Escrita):

Análise de Operações

- **Read-Heavy:** O sistema lê muito mais do que grava. (Ex: Catálogo de Filmes).
  Estratégia: Duplicar dados para ler em 1 query.
- **Write-Heavy:** O sistema grava em alta velocidade. (Ex: Logs de Sensores).
  Estratégia: Documentos simples e escrita rápida.

Embutir (Embedding)

Colocar o dado dentro do documento principal. **Quando usar?**

- Relacionamentos 1:1 ou 1:Poucos.
- Dados que são lidos juntos quase sempre.
- Dados que mudam raramente.

Exemplo: Endereço dentro de Usuário

```
{
  "nome": "Jo o Silva",
  "endereco": { "rua": "Av Paulista", "cidade": "SP" }
}
```

Referenciar (Referencing)

Guardar o ID de outro documento (como uma Foreign Key). **Quando usar?**

- Relacionamentos 1:Muitos (Milhares de itens).
- Dados que mudam com alta frequência em múltiplos lugares.
- Para evitar documentos maiores que 16MB.

Exemplo: Alunos em um Curso

```
{
  "titulo": "Curso de MongoDB",
  "alunos_ids": [ObjectId("507f1f"), ObjectId("223f2g")]
}
```

Cardinalidades

- **1:1 (Perfil de Usuário):** Sempre **Embed**. Não há motivo para separar o CPF do nome se eles sempre aparecem juntos.
- **1:N (Post e Comentários):**
  - Se o post tem poucos comentários: **Embed**.
  - Se o post pode ter milhões (ex: post viral): **Reference** (Crie uma collection ‘comentarios‘).
- **N:N - Livros e Autores** No MongoDB, usamos o padrão de **Array de Referências nos dois lados**:
  - O documento do Livro contém um array de `autores_ids`.
  - O documento do Autor contém um array de `livros_ids`.

  Isso permite encontrar todos os livros de um autor e todos os autores de um livro sem tabelas de junção.